

JAVASCRIPT CHEATSHEET

VAR VS LET VS CONST

var let const						
var let and const are all keywords that declare variables.						
	global scoped	function scoped	block scoped	reassignable	redeclarable	can be hoisted
var	+	+	-	+	+	+
let	-	+	+	+	-	-
const	-	+	+	-	-	-

Important: Let a aparut dupa var pentru a face variabilele mai previzibile, pentru ca are un comportament mai usor de anticipat fata de var.

```
const firstName = "John";
```

```
var age = 25;
```

```
let isStudent = false;
```

```
console.log(`Name: ${firstName}, Age: ${age}, Student: ${isStudent}`);
```

In JavaScript datele sunt fie primitive fie date prin referinta.

Tipurile de date primitive: string, number, boolean, etc.

Tipurile de date prin referinta: arrays, functii, obiecte.

Tipurile de date primitive sunt immutable iar cele prin referinta sunt mutable.

Hoisting este comportamentul prin care declaratiile de variabile si functii sunt mutate in varful **scopului** care le contine, la compilare.

```
console.log(myVariable); // Output: undefined
```

```
var myVariable = 10;
```

```
hoistedFunction(); // Output: "Function is hoisted!"
```

```
function hoistedFunction() {  
    console.log("Function is hoisted!");  
}
```

```
// Example with let
```

```
console.log(myLetVariable); // This would result in a  
ReferenceError
```

```
let myLetVariable = 5;
```

```
console.log(myLetVariable); // Output: 5
```

Destructuring sintaxa care despachetează valorile din array-uri sau obiecte în variabile distincte.

```
const [a, b] = array;  
const { a, b } = obj;  
const { new_name: obj_field_name, new_name_2: field_2 } = obj;  
const [a, b, ...rest] = array;  
const { a: a1, b: b1, ...rest } = obj;
```

```
obj = {a: "new message", ...obj}; // modificăm un singur field
```

JSON - JAVASCRIPT OBJECTS

```
// Creating an object  
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 30,  
  sayHello: function() {  
    console.log('Hello, my name is ${this.firstName}  
    ${this.lastName}.');  
  }  
};  
  
// Accessing object properties  
console.log("First Name:", person.firstName); // Outputs: John  
console.log("Age:", person.age); // Outputs: 30  
  
// Calling object method  
person.sayHello(); // Outputs: Hello, my name is John Doe.
```

Fetch API este o interfață modernă în JavaScript utilizată pentru a efectua cereri HTTP.

```
async function logMovies() {
```

```
const response = await
fetch("http://example.com/movies.json");
const movies = await response.json();
console.log(movies);
}
```

Putem face fetch() si pentru alte [requesturi HTTP](#).

```
const apiUrl = 'https://jsonplaceholder.typicode.com/todos/1';

fetch(apiUrl, {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ completed: true }),
})
.then(response => response.json())
.then(updatedData => console.log('Updated Data:',
updatedData))
.catch(error => console.error('Fetch error:', error));
```